

USER MANUAL

Delivery Navigation ROS 2 System

1. Introduction

The Delivery Navigation system is a ROS 2 application that simulates a delivery robot moving toward a destination.

The system consists of three main nodes:

- **Navigation Publisher:** publishes the robot's current navigation status.
- **Navigation Subscriber:** receives the robot status, displays warnings, and requests the estimated arrival time.
- **Navigation Service Server:** provides a service that calculates the estimated arrival time based on the remaining distance and robot speed.

The project is implemented using **ROS 2 Humble**, **Python**, and an **OOP-based structure**.

2. Requirements

Before running the project, make sure the following are installed:

- Ubuntu 22.04
- ROS 2 Humble
- Python 3
- Colcon
- Git

The workspace used in this project is:

~/ros2_workspace

3. Project Setup

Open a terminal and navigate to the workspace:

```
cd ~/ros2_workspace
```

ROS 2 Humble:

```
source /opt/ros/humble/setup.bash
```

Build the project:

```
colcon build
```

After building, source the workspace:

```
source install/setup.bash
```

4. Running the System

The complete system can be launched using the launch file:

```
ros2 launch delivery_navigation navigation_launch.py
```

This command starts the following nodes:

- nav_publisher
- nav_service_server
- nav_subscriber

The publisher starts with a default remaining distance of **200 meters** and a speed of **2 m/s**.

The distance decreases as the robot moves.

5. Expected Output

The subscriber displays the robot status and estimated arrival time.

Example:

Distance: 200.0m | Speed: 2.0m/s

Estimated arrival time: 1.67 minutes

Distance: 198.0m | Speed: 2.0m/s

Estimated arrival time: 1.65 minutes

Distance: 196.0m | Speed: 2.0m/s

Estimated arrival time: 1.63 minutes

The estimated arrival time is calculated using:

$\text{Time} = \text{Distance} / \text{Speed} / 60$

For example:

$200 \text{ m} / 2 \text{ m/s} = 100 \text{ seconds}$

$100 / 60 = 1.67 \text{ minutes}$

6. Obstacle Warning

The navigation publisher randomly generates obstacle information.

When an obstacle is detected, the subscriber displays a warning such as:

WARNING: Obstacle detected! Distance: 184.0m

When no obstacle is detected, the subscriber displays the normal navigation information.

7. Testing the Arrival Time Service Manually

The arrival time service can also be tested independently.

Run the service server: `ros2 run`

`delivery_navigation nav_service_server`

Then, in another terminal, call the service:

```
ros2 service call /estimate_arrival_time robot_interfaces/srv/EstimateArrivalTime
"{distance_remaining: 20.0, speed: 2.0}"
```

Expected result:

`minutes_remaining: 0.16666667`

This means that a robot with 20 meters remaining and a speed of 2 m/s will need approximately **0.17 minutes** to reach the destination.

8. Stopping the System

To stop the running nodes, press:

`Ctrl + C`

9. Troubleshooting

ROS 2 command not found

Make sure ROS 2 Humble is sourced:

```
source /opt/ros/humble/setup.bash
```

Package not found

Source the workspace:

```
source ~/ros2_workspace/install/setup.bash
```

Changes are not detected

Rebuild the workspace:

```
cd ~/ros2_workspace
```

```
colcon build source
```

```
install/setup.bash
```

Arrival time service is not available

Make sure the service server is running or start the complete system using:

```
ros2 launch delivery_navigation navigation_launch.py
```

10. Conclusion

The Delivery Navigation system can be started using a single launch command. It continuously publishes robot navigation information, detects obstacles, and calculates the estimated arrival time in minutes through a ROS 2 service.

The system is designed using object-oriented principles to separate the navigation logic, status processing, and service logic from the ROS 2 nodes.